For the final project, you will work in teams to design, build, and deploy a fully automated, production-grade Machine Learning Operations (MLOps) pipeline. Utilizing the knowledge and tools acquired throughout this course, each group will select a real-world dataset, build an automated training pipeline, deploy the model as a reproducible service, and establish a continuous monitoring framework to detect operational failures.

Project Milestones & Timeline

- **Group Formation & Topic Selection (Due: End of Session 4):** Students must form groups (exact group sizes will be announced in class based on enrollment). Teams must submit their selected dataset, problem statement, and group roster to the instructor via email. Topics are approved on a **first-come, first-served** basis to prevent project replication.
- **Project Proposal (Due: End of Session 4):** Along with your topic submission, include a brief project proposal (**up to 300 words**) outlining the nature of the dataset, the predictive problem statement, and an architectural sketch of your proposed MLOps toolstack.
- **Final Presentations (Session 10):** Each group will have **10–15 minutes** to orally present their project and answer questions from the instructor and peers.

Individual Contribution: During the presentation, each group member must explicitly state their distinct engineering contributions to the project.

Technical Project Outline

Your team's pipeline must implement the following core stages of the MLOps lifecycle:

1. **Data Ingestion & Baselines**
    - Select a dataset with a clear target/outcome variable.
    - Establish an appropriate evaluation metric aligned with the business/problem constraints.
    - Perform a train/test split, ensuring the test set remains isolated until production validation.
2. **Pipeline Automation & Experiment Tracking**

- Build an automated orchestrator pipeline using **Airflow**, **Prefect**, or an equivalent workflow platform to automate data preprocessing and training.
- Implement experiment tracking and model logging using **MLflow** (or a similar tool like Weights & Biases) to manage hyperparameters, artifacts, and metrics (e.g., using AutoML to find the optimal algorithm).
- Log the finalized model to a **Model Registry** with proper semantic versioning.

3. **Containerization & Deployment**
    - Package the registered model and deploy it for inference (e.g., using Docker paired with FastAPI, Flask, or BentoML).
    - The deployment should accept test inputs and return real-time predictions.
4. **Production Monitoring & Drift Simulation**
    - Set up a model monitoring framework or dashboard (e.g., EvidentlyAI, Prometheus/Grafana, or a custom dashboard) to track data and model performance.
    - **Baseline Validation:** Pass the clean test dataset through your deployed API and validate the results against your monitoring baseline.
    - **Stress-Test / Drift Simulation:** Artificially corrupt the test dataset to simulate data or concept drift (e.g., introduce out-of-bounds random values, swap feature columns, or alter data schemas).
    - **Anomaly Verification:** Send this "drifted" data to your deployed model. Document and verify how your monitoring dashboard catches and alerts you to this system anomaly.

Deliverables

1. **GitHub Repository**

Your repository must be clean, modular, and professional. It must include:

- Fully commented source code.
- A comprehensive README.md explaining how to run and reproduce the entire pipeline locally.

    - A dependency configuration file (requirements.txt, environment.yml, or a Dockerfile).
2. **Presentation Slides (PPT)**

Your presentation must cover the following structural points:

- **Problem Statement & EDA:** Overview of the chosen dataset and problem constraints.
- **Evaluation Metric:** Justification for your chosen success criteria.
- **System Architecture:** A visual diagram of your training and deployment pipelines.
- **Experimentation Tracking:** Your MLflow/tracking dashboard showing algorithm selection.
- **Deployment & Monitoring:** A live demo or screenshots of your containerized API and monitoring dashboard.
- **Drift Analysis:** Visual evidence of your monitoring system responding to the corrupted test data.
- **Repository Link:** A visible link to your public/accessible GitHub repository.